DHAR

Bare Metal. Absolute Control.

CANON AMENDMENT A-1

AMENDMENT A-1

"Security first, then artificial intelligence, then quantum computing."

DOCUMENT	Amendment A-1 — Direction
VERSION	1.0
DATE	October 26, 2026
STATUS	Ratified
AMENDS	Spec0, Spec1, Stage-alpha to Stage-epsilon, the Volume III documents, The Pact (the Master Index, §8.2, lists the changes)
KIND	Forward-looking specification, in the voice of the Canon: it states what Dhar is to become and the tests that say when it has
PART OF	The Canon of Dhar, Amendments
COMPANION	Amendment A-2 — Stage 0 Targets
AUTHOR & PROJECT LEAD	Abdullahi Baba Garba

Contents

1. Why this amendment exists 3

2. The direction in one paragraph 3

3. Principles that hold across all three 3

4. Track S: Cybersecurity (first) 4

5. Track A: Artificial intelligence (second) 4

6. Track Q: Quantum computing (third) 5

7. Order and dependencies 5

8. What Stage 1 must leave room for now 6

9. Acceptance gates 7

10. Non-goals and what Dhar will not claim 8

11. Ratification 8

1. Why this amendment exists

The Canon already names the domains Dhar is for: systems, hardware, backend and web, cybersecurity, data and AI, blockchain. It does not say which comes first, and it does not say how anyone will know that Dhar has become good at any of them. This amendment does both.

It is not a change of course, and it removes nothing. The founding ambition stands: C-class performance, Java-class safety, Python-class readability, complete independence from LLVM, the JVM and any outside runtime, and a language and toolchain built in Nigeria for Nigerian and African technological independence. Every use case the Canon already names stays: systems and hardware, backend and web, data, blockchain and Web3, and the rest. What is added is an order of attack, a set of design requirements that must be built into the compiler early so they are not bolted on late, and a test for every claim.

2. The direction in one paragraph

Dhar is a systems language for **trustworthy computation**. It goes first to software where a mistake is an attack: cryptography, protocols, parsers, forensic and defensive tools, anything that handles hostile input. It goes second to the systems software under artificial intelligence: inference engines, tokenisers, data pipelines, and small models that must run on modest hardware. It goes third to software for the quantum era: cryptography that survives a quantum computer, and the description and simulation of quantum programs. The order is chosen because each step uses what the one before it built. The order is a statement about what is built and tested first, never about what Dhar is for: the other domains are served by modules on the same core (P6).

3. Principles that hold across all three

- **P1. Independence.** The core language, compiler and standard library depend on no outside compiler, runtime or library. Outside implementations (OpenSSL, libsodium, NumPy, Qiskit and the like) may be used as *test oracles* to check Dhar's answers, never as dependencies.
- **P2. Safety by construction.** Every index is checked. Ownership and borrowing are enforced at compile time. There is no garbage collector. The places where safety is deliberately given up (`asm`, `foreign`, `volatile`, raw buffers) are explicit, small, and findable by search.
- **P3. Every claim has a gate.** A statement about Dhar's security, speed or correctness is allowed only if a test against a named reference passes. The gates are in section 9.
- **P4. A small, auditable path from source to executable.** Few stages, readable code, reproducible output, a recorded list of what went into every build.
- **P5. Nothing retrofitted.** What the three tracks need from the compiler (section 8) is designed into the first real intermediate representation, even though the features themselves arrive later.
- **P6. Breadth is kept, as modules.** The three tracks decide the order of work and where acceptance gates are written first. They do not remove, narrow or delay any other use case. Systems and hardware work, backend and web development, data processing, blockchain and Web3, mobile and every other domain the Canon names are

delivered as modules on Kun and the standard library, built on the same core and held to the same safety floor (P2), so the language stays modular, robust and able to do what its users need. A module outside the three tracks has no acceptance gate in this amendment; a later amendment may add one, and a module is never held back because a gate has not been written for it.

4. Track S: Cybersecurity (first)

Why Dhar fits. No libc and no runtime means a small attack surface. Checked indexing removes the commonest memory error. A compiler written in a few thousand lines of assembly, and later in Dhar itself, is small enough for a person to audit. A language that targets the hardware directly can control what the machine code does with a secret.

S1. Memory safety floor. Every indexed access is checked at run time or proved safe at compile time. Ownership and borrowing (`view`, `grab`, as in Spec1) are enforced by the compiler, so safe code cannot use memory after it is freed, free it twice, or alias a mutable value. There is no collector and no hidden allocation.

S2. Constant-time code. A value marked *secret* may not decide a branch, may not be an array index, and may not be the operand of an instruction whose timing varies with its value (division is the usual example). The compiler rejects such code, and the optimiser is forbidden to create it (section 8, R2). The only ways to turn a secret into a public value are an explicit, auditable *reveal*, and operations whose result is a mask or a constant-time select. Secret buffers are cleared when they go out of scope, and the clearing may not be optimised away. The keyword and syntax are proposals for Spec1 (see the index); the semantics above are the requirement.

S3. A first `std.crypto`, written in Dhar. SHA-256 (FIPS 180-4), SHA-3 and SHAKE (FIPS 202), ChaCha20-Poly1305 (RFC 8439), X25519 (RFC 7748) and Ed25519 (RFC 8032). Each passes the published test vectors, and each secret-handling routine passes a timing test (section 9) showing no input-dependent variation.

S4. A build you can trust.

1. *Fixed point.* The compiler built by Stage 0 and the compiler that compiler builds are byte-for-byte identical.
2. *Two independent compilers.* Stage 0 is written in assembly and Stage 1 in Dhar, so they are independent implementations. Stage 0 compiles the Stage 1 source, that result compiles it again, and the second result must equal the Stage 1 binary that is distributed. Equality shows the distributed binary came from the published source and carries no hidden change (the check known as diverse double-compilation), unless Stage 0 itself is compromised. This requires Stage 0 to be kept frozen and archived, not improved, after the hand-off.
3. *Reproducible.* Two machines, same source, same flags, identical output.
4. *Listed.* Every build emits a software bill of materials, and releases are signed.

5. Track A: Artificial intelligence (second)

Why Dhar fits. A trained model is numbers in arrays and a fixed sequence of operations on them. A language with checked bounds, no garbage collector and no runtime can run it with memory use known before it starts and with no pauses. Efficient inference matters most where compute, power and bandwidth are scarce, which describes much of the market Dhar is built for.

A1. Numbers for models. `f32` and `f64`, then `f16` and `bf16`; fixed-point and 8-bit integer arithmetic with explicit wrapping, saturating and checked variants; fixed-width SIMD vector types that map to the vector unit of each target. Floating-point follows IEEE 754 and nothing is reordered to be faster unless the programmer asks for it.

A2. Tensors without a garbage collector. Fixed-shape arrays, and views over them with checked bounds; arena (region) allocation, so a model's whole memory use is one allocation whose size is known before it runs.

A3. Inference end to end. A small trained network (a handwritten-digit classifier is the reference) runs from a Dhar program with no outside runtime. Weights come from a documented, simple file format. Its outputs match a reference framework to a stated tolerance on a fixed test set.

A4. Speed, measured. The gap to optimised C on matrix multiplication and on that network is measured and published, whatever it turns out to be. Accelerators (GPUs and similar) and training are outside this track until A4 is met.

6. Track Q: Quantum computing (third)

Two separate needs share this track.

Q1. Post-quantum cryptography. ML-KEM (FIPS 203), ML-DSA (FIPS 204) and SLH-DSA (FIPS 205), written in Dhar, each passing the NIST known-answer tests. This protects data today against a quantum computer built later, and needs no quantum hardware. It depends only on S3 (it uses SHA-3 and SHAKE) and so may be scheduled as soon as S3 passes, ahead of the rest of the track.

Q2. Circuits as data. A Dhar program builds a quantum circuit (qubit registers, gates, measurement, classical control) as an ordinary value and writes it out as OpenQASM 3, which a reference tool accepts and simulates to the expected result. No new syntax is needed for the first version; it is a library.

Q3. A state-vector simulator in Dhar. It needs complex numbers (a `c128` type built on A1). Memory doubles with every qubit: 16 bytes per amplitude means 16 MiB at 20 qubits, 512 MiB at 25 and 16 GiB at 30, so the simulator states its limit and refuses past it. Results match a reference simulator on a fixed set of circuits.

Q4. Hardware. Driving real quantum hardware is outside this amendment until Q2 and Q3 pass and a provider offers an interface that Dhar can use without an outside runtime.

7. Order and dependencies

Nothing in sections 4 to 6 starts before Stage 1 compiles itself, and Stage 1's self-hosting work begins only when Stage 0 is complete on every target (Amendment A-2). Until then the work is Stage 0 and the design requirements of section 8, and only those. After that, the order is the order of the tracks, with these dependencies:

GATE	NEEDS FIRST
S1	Stage 1 type checker, borrow rules (Spec1)
S2	Stage 1 IR with the secret attribute (R2)

GATE	NEEDS FIRST
S3	S1, S2; integer types of every width; rotates and carry operations
S4	Stage 1 self-hosting; Stage 0 archived
A1	Floating-point in the IR and in at least one back end
A2	S1 (views, borrowing); arena allocation
A3	A1, A2; file input
A4	A3; a register allocator (Stage 2)
Q1	S3
Q2	Collections and strings in the standard library
Q3	A1; complex type; Q2
Q4	Q2, Q3

8. What Stage 1 must leave room for now

These are design requirements, not features. They are cheap to include in the first intermediate representation and expensive to add after the optimiser exists.

- **R1. Typed values wide enough for the tracks.** The IR's type set includes integer widths from 8 to 128 bits, `f16`, `bf16`, `f32`, `f64`, fixed-width vectors and a complex pair, even though Stage 1's front end does not yet produce them.
- **R2. A secret attribute that the optimiser must respect.** Values and memory regions can be marked secret, and the IR records it. Stage 2's passes (constant folding, strength reduction, if-conversion, loop transforms, inlining) may not turn a select into a branch on a secret, may not introduce a table lookup indexed by one, and may not delete the clearing of a secret buffer.
- **R3. No hidden allocation.** Every allocation is visible in the IR and belongs to a region, so memory use can be bounded and arenas are natural.
- **R4. Targets described by tables.** The IR's back ends read a description of each processor: vector width, crypto instructions (AES, SHA, carry-less multiply, the ARM and RISC-V cryptography extensions), alignment rules. A new processor is a new table and a small emitter, not a new compiler. Stage 0 reaches the same targets without an intermediate form (Amendment A-2); Stage 1's back ends are re-implemented on the IR and must pass the same tests.
- **R5. Stage 1 stays inside what Stage 0 accepts** until the hand-off, as the bootstrap requires. The tracks add nothing to Stage 1's scope except R1 to R4.

9. Acceptance gates

A gate is passed only when its test runs in the repository and passes against the named reference.

GATE	TEST	REFERENCE (ORACLE)	PASS
S1	A suite of programs that index out of range, use after free, double free and alias mutably	Expected rejection or trap, per Spec1	Every program is rejected at compile time or traps; none corrupts memory
S2	Programs that branch on, index with, or divide by a secret; programs that copy a secret through the optimiser	Expected rejection	All rejected; optimised output has no secret-dependent branch or access (checked by inspection of the emitted code and by the timing test below)
S3	Known-answer vectors for every algorithm; then a timing test: run each secret-handling routine on two input classes at least a million times each and compare the timing distributions	FIPS 180-4, FIPS 202, RFC 8439, RFC 7748, RFC 8032 vectors; a published statistical timing method	All vectors pass; the timing test finds no significant difference
S4	Build the compiler twice on one machine and once on a second	Byte comparison; bill-of-materials check	All three outputs identical; bill of materials complete
A1	Elementary and fused operations on edge values (zero, infinity, not-a-number, subnormals, rounding ties)	The platform's IEEE 754 results; a reference library	Bit-identical for every operation the language promises to be exact
A2	Programs that index tensors out of range and misuse views	Expected rejection or trap	None corrupts memory; arena size known before run
A3	Run the reference network on a fixed test set	A reference framework's output	Predictions agree to the stated tolerance on every test input
A4	Matrix multiply and the A3 network	Optimised C (`-O3`) on the same machine	The ratio is measured, recorded with the method, and published as it stands
Q1	NIST known-answer tests for each parameter set	NIST ACVP and the published KAT files	All pass
Q2	A fixed set of circuits	A reference tool's parse and simulation	The tool accepts every file; simulated results match the expected ones
Q3	The same circuits, up to the stated qubit limit	A reference simulator	Amplitudes agree to a stated tolerance; the simulator refuses past its limit
Q4	Not defined until Q2 and Q3 pass		

10. Non-goals and what Dhar will not claim

- Dhar is not formally verified, and nothing here says it will be.
- Constant-time is a property of the instructions Dhar emits on an instruction set whose timing is documented. It is not a promise about every microarchitectural side channel.
- No GPU or accelerator support, no model training, no quantum hardware drivers, within this amendment.
- No public statement says "secure", "safe" or "fast" without naming the gate that backs it. The allowed form is "passes gate S3 on these vectors as of this commit".
- The same rule holds for targets: a statement that Dhar runs on a processor or operating system names the target's gate in Amendment A-2 (section 6 there) and the run that passed it.

11. Ratification

This amendment takes effect when the project lead ratifies it. Each document named in the Master Index (§8.2) is then amended in the same pass, as the Canon's working principle requires: a correction goes into every affected document, not only the first.

Ratified by	Abdullahi Baba Garba, project lead
Date	October 25, 2026 (instruction to apply the amendment to every Canon document); issued as v1.0 on October 26, 2026
Notes	The order of the tracks (security, then AI, then quantum) and principle P6 are the author's. The semantics of the secret qualifier, the wide numeric types and the named system calls are ratified here; their keyword and syntax spellings are proposals in Spec1 v3.2 and are marked provisional there until the author fixes them.

Revision log

Amendment A-1 — Direction · v1.0 · October 26, 2026 · Ratified. Contains no status: what exists today is in the Progress Reports, which are outside the Canon.

VERSION	DATE	CHANGE
v0.1	October 22, 2026	Drafted as a proposal: direction, principles P1–P5, tracks S, A and Q, design requirements R1–R5, acceptance gates.
v1.0	October 26, 2026	Ratified on the author's instruction of October 25, 2026 to apply every amendment to every Canon document. Principle P6 added (breadth is kept, as modules); §1, §2, §7, §8 and §10 aligned with Amendment A-2 and with P6; ratification block completed.